Regex Crash Course

Anchors

`^` - Notates the absolute start of a line

`$` - Notates the absolute end of a line

Conditioners

> **Note**
>
> The `|` symbol you see between the 1st and 2nd square bracket group is known as a *vertical pipe* and can be entered by pressing `[SHIFT]` + `\` simultaneously on your keyboard in most cases.

`|` - Conditions "OR"; for example, `a|b` is "a OR b".

Basic Selectors

> **Important**
>
> You need to escape certain special characters that Regex usually interprets with *backslashes* `\` to interpret them literally (i.e. `\.` matches a literal period)
> such as parenthesis `()`, square brackets `[]`, curly brackets `{}`, the up caret `^`, or a literal backslash `\`, etc.

> **Note**
>
> Selectors with a **`!!!`** after them can be capitalized to invert their meaning;
> i.e. `\S` matches any *non*-whitespace character.

`.` - That's a period, it matches all characters with the sole exception of a newline.

`\s` - **`!!!`** A backslashed lowercase `s` which matches all *whitespace* characters.

`\w` - **`!!!`** A backslashed lowercase `w` and is the equivalent of `[a-zA-Z0-9_]` which matches any character that is a standard English alphabetical character, a number, or underscore `_`.

`\d` - **`!!!`** A backslashed lowercase `d` and is the equivalent of `[0-9]` which matches any character which is a number.

`\b` - **`!!!`** Indicates a word boundary; used to look for the explicit end or start of a word usually.

- Example: `\bTEST\b` matches exactly the word `TEST` with no other word characters around it.

`[` and `]` - Notates the start and end of a *"range"*.

`[x1@]` - Match a character that is x, 1 or @.

`[^x1@]` - Match any character that is NOT x 1 or @.

`[a-z]` - Match any lowercase character a through z.

`[A-Z]` - Match any uppercase character A through Z.

> **Note**
>
> You usually avoid using `[A-z]` as it it contains non-alphabetical characters between uppercase `Z` and lowercase `a`.
> You also avoid using `[a-Z]` because lowercase characters come *after* uppercase characters in the ASCII table.

`[0-9]` - Match any numerical character 0 through 9.

`[a-z1-3]` - Yeah, you can do multiple ranges in 1 square bracket group.

`(` and `)` - Notates the start and end of a capturing group--good for isolating tokens and readability, especially when using the OR conditioner.

- Example: `(^[abc]|[xyz]$)` matches any string which starts with a, b or c... OR matches any string ending in x, y or z.

Quantifiers

`?` - 1 or none of the preceding selector.

$\{n_1,n_2\}$ - n_1 to n_2 amounts of the preceding selector, where n_1 and n_2 are *non-negative integers*.

Greedy Quantifiers

> **Note**
>
> These guys like to take as much as possible.
>
> Say if you wanted everything up until the **first** forward slash `/`, you would *not* use `.+/`.
> You would either use `[^/]+`, which selects anything that isn't a forward slash or `.+?/`; which selects the fewest amount of any characters preceding a forward slash.

`*` - None to infinite amounts of the preceding selector.

`+` - 1 to infinite amounts of the preceding selector.